PROYECTO 7 / SISTEMAS OPERATIVOS

Cuando un hilo debe esperar.

Implementación de monitores con mutex y variables de condición.

C17 | Ubuntu Linux | POSIX Threads | Sin semáforos

Completar integrantes, institución, grupo y docente antes de entregar.

01 / EL PROBLEMA

Compartir un recurso no significa usarlo sin reglas.



Búfer lleno

El productor debe esperar espacio. No sobrescribir elementos pendientes.

Búfer vacío

El consumidor debe esperar datos. No extraer información inexistente.

Un monitor reúne estado, operaciones y reglas.

Exclusión mutua

Un mutex protege celdas, índices y contadores.

C17 no incorpora monitores: los construimos mediante encapsulación.

Espera por condición

`not_full`: espacio.
`not_empty`: datos.

La condición no almacena permisos como un semáforo.

Despertar no basta.
Hay que volver a comprobar.

```
lock(mutex);
while (count == capacity && !closed)
    cond_wait(not_full, mutex);
// comprobar cierre, insertar y actualizar estado
cond_signal(not_empty);
unlock(mutex);
```

`wait` libera el mutex y lo recupera al retornar.
`signal` no entrega inmediatamente el mutex.

04 / DEMO A

Productor bloqueado.

1. Iniciar únicamente al productor y llenar el búfer.
2. Observar **WAIT / not_full**: la siguiente inserción espera.
3. Enter inicia al consumidor: extrae y señala.

WAIT P → GET C → SIGNAL C → WAKE P → PUT P

```
./build/monitor-demo --scenario producer --capacity 5 --items 12
```

05 / DEMO B

Consumidor bloqueado.

1. Iniciar únicamente al consumidor con búfer vacío.
2. Observar **WAIT / not_empty**.
3. Enter inicia al productor: inserta y señala.

WAIT C → PUT P → SIGNAL P → WAKE C → GET C

```
./build/monitor-demo --scenario consumer --capacity 5 --items 12
```

06 / CORRECCIÓN Y OBSERVACIÓN

La UI muestra el monitor. No lo reemplaza.

Invariantes

```
0 ≤ count ≤ capacity
producidos − consumidos = count
```

FIFO; índices circulares; todos los datos bajo lock.

Snapshots + eventos

Copiar con lock; dibujar sin lock.
Sin sleeps dentro de la sección crítica.

Historial acotado: puede omitir eventos visuales a máxima velocidad.

07 / EVIDENCIA LOCAL

Verificado, no solo observado.

160 000

transferencias por suite de estrés
4 capacidades × 4 repeticiones × 10 000

FIFO y wraparound
Ambos bloqueos y reanudaciones
Fallos de creación de hilos
Salida, colores y restauración de terminal

`make check · make sanitize · make tsan`

Resultados locales, 10/10/2026. ASan/UBSan fuera del sandbox; TSan independiente. No son una prueba exhaustiva de todas las intercalaciones.

08 / CIERRE SEGURO

Terminar también es parte de la sincronización.

closed = true → broadcast ambas → pthread_join → destruir recursos

Finalización normal: drenar el búfer.
Cancelación: detener y reportar `CANCELLED`.

No usar pthread_cancel. No destruir un mutex mientras un hilo pueda necesitarlo.

09 / CONCLUSIÓN Y FUENTES

Esperar correctamente también es progresar.

Mutex para exclusión. Condiciones para coordinación. Encapsulación para razonar. Evidencia para demostrar.

Hoare, “Monitors”, CACM 17(10), 1974. DOI: 10.1145/355620.361161
POSIX: wait y signal/broadcast
GCC: soporte de estándares C

¿Por qué while? ¿Qué libera wait? ¿Qué garantiza signal?